COMPUTER VISION

LABSHEET-5

NAME-HARSH RAJ

CU ID-240251762

#2. Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import data, color, transform
from skimage.color import rgb2gray
```

#3. Load a Real-World Image

```
# Load a real-world image
image = data.astronaut()

# Convert RGB to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

# Basic preprocessing
gray = cv2.GaussianBlur(gray, (3, 3), 0)

# Create a similar image by rotating the original
h, w = gray.shape
```

```
center = (w // 2, h // 2)

rotation_matrix = cv2.getRotationMatrix2D(center, 15, 1.0)
gray2 = cv2.warpAffine(gray, rotation_matrix, (w, h))

# Display images
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray2, cmap='gray')
plt.title("Similar Rotated Image")
plt.axis("off")

plt.show()

#4. Apply SIFT to Detect Keypoints and Compute Descriptors
# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)
```

```
print("Number of SIFT keypoints:", len(keypoints))
print("Descriptor shape:", descriptors.shape)

#5. Visualize SIFT Keypoints
# Draw keypoints on the image
sift_image = cv2.drawKeypoints(
    gray,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(8, 6))
plt.imshow(sift_image, cmap='gray')
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()

#6. Extract HOG Features

# Normalize image to [0, 1]
gray_float = gray.astype(np.float32) / 255.0

# Extract HOG features
hog_features, hog_image = hog(
```

```
    gray_float,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm='L2-Hys',
    visualize=True
)

print("HOG feature vector length:", len(hog_features))

#7. Visualize HOG Descriptor
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray')
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image, cmap='gray')
plt.title("HOG Visualization")
plt.axis("off")

plt.show()

#8. Analyze SIFT Keypoint Distribution
```

```
# Divide image into 4 regions
regions = {
    "Top-Left": gray[:h//2, :w//2],
    "Top-Right": gray[:h//2, w//2:],
    "Bottom-Left": gray[h//2:, :w//2],
    "Bottom-Right": gray[h//2:, w//2:]
}

# Count keypoints in each region
region_counts = {}

for name, kp in zip(["Top-Left", "Top-Right", "Bottom-Left", "Bottom-Right"],
                    [[], [], [], []]):
    region_counts[name] = 0

for kp in keypoints:
    x, y = kp.pt

    if x < w//2 and y < h//2:
        region_counts["Top-Left"] += 1
    elif x >= w//2 and y < h//2:
        region_counts["Top-Right"] += 1
    elif x < w//2 and y >= h//2:
        region_counts["Bottom-Left"] += 1
    else:
        region_counts["Bottom-Right"] += 1
```

```
print(region_counts)


#9. Basic Image Matching Using SIFT


# Detect SIFT features in both images
kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(gray2, None)
# FLANN matcher
index_params = dict(algorithm=1, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
# Find the two nearest matches
matches = flann.knnMatch(des1, des2, k=2)
# Apply Lowe's ratio test
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
print("Total matches:", len(matches))
print("Good matches:", len(good_matches))


#10. Visualize the Matches
match_image = cv2.drawMatches(
    gray, kp1,
    gray2, kp2,
```

```
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
plt.figure(figsize=(14, 6))
plt.imshow(match_image, cmap='gray')
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()
```

OUTPUT:-













QUESTION AND ANSWER

1. What is feature extraction, and why is it important in computer vision?

Feature extraction is the process of identifying important information such as edges, corners, shapes, and textures from an image. It reduces complex image data into useful features for recognition, matching, and classification.

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

SIFT detects distinctive keypoints at different scales, assigns orientations, and generates 128-dimensional descriptors. These descriptors are matched between images to recognize the same object despite changes in size or rotation.

3. What are keypoints and feature descriptors in image analysis?

Keypoints are distinctive locations in an image, such as corners or textured regions. Feature descriptors are numerical representations of the local image information around those keypoints.

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

HOG divides an image into small cells and calculates the distribution of gradient orientations in each cell. It is significant because it captures object shape and edge structure, making it useful for object detection.

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

SIFT is more robust to scale and rotation changes but is generally computationally more expensive. HOG is relatively efficient and works well for shape-based detection, but is more sensitive to image alignment.

6. Why is SIFT considered invariant to scale and rotation?

SIFT detects features at multiple scales and assigns a dominant orientation to each keypoint. This allows the descriptors to remain similar even when the image is resized or rotated.

7. Mention three real-world applications where HOG descriptors are commonly used.

Pedestrian detection ,Vehicle detection ,Face detection

8. Why is feature extraction performed before image classification or object detection?

Feature extraction converts raw image data into meaningful information such as edges, shapes, and textures. This makes classification and detection faster, easier, and more accurate.

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Advantages: They are interpretable, require less training data, and can be computationally efficient.
Limitations: They may perform poorly on complex images and require manual selection of features, whereas deep learning can automatically learn more powerful representations.

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

They provide distinctive image information that helps systems match similar images, recognize faces, and detect objects. SIFT is useful for local matching, while HOG is effective for shape-based detection.